Université du Québec à Chicoutimi

LIARA - Laboratoire sur l'Intelligence Ambiante pour la Reconnaissance d'Activité

Stage - Juin à Décembre 2026

Compréhension des données Unitree LiDAR L2

Étudiant
BONNEAU Mathieu

Remis à
Julien MAITRE

20 juillet 2026

UQAC
Université du Québec
à Chicoutimi

Table des matières

1 Acquisition et données **2**

2 Enregistrement ROS 2 **2**

3 Deux méthodes d'accès **2**

4 Approche recommandée pour le temps réel **3**

5 Conclusion **3**

1 Acquisition et données

Le Unitree L2 transmet ses mesures par Ethernet ou par port série. Le SDK C++ Unitree décode les paquets du capteur et produit deux ensembles principaux de données [**unitree_sdk**] :

Donnée	Sujet ROS 2	Contenu principal
Nuage de points	`/unilidar/cloud`	`x`, `y`, `z`, `intensity`, `ring` et `time`
IMU	`/unilidar/imu`	Quaternion (x, y, z, w), vitesse angulaire et accélération linéaire

Dans le message ROS 2 `PointCloud2`, les coordonnées, l'intensité et le temps relatif sont généralement des `float32` ; `ring` est un `uint16`. Chaque message possède également un horodatage et un repère `frame_id`. Le temps d'un point peut être estimé par :

$$t_i = t_{\text{nuage}} + \Delta t_i. \tag{1}$$

Les repères sont importants si le capteur se déplace ou si les points doivent être projetés sur une caméra. Il est donc utile d'enregistrer aussi `/tf` et `/tf_static` lorsqu'ils sont disponibles.

2 Enregistrement ROS 2

Un bag ROS 2 contient un fichier `metadata.yaml` et un ou plusieurs fichiers `.db3`. Ces derniers sont des bases SQLite contenant les messages sérialisés en CDR. Les données ne sont pas chiffrées, mais leur forme binaire doit être désérialisée avant d'obtenir les nombres.

ROS 2 réalise automatiquement cette opération pendant `ros2 bag play`. Un nœud reçoit alors les mêmes messages `PointCloud2` et `Imu` qu'avec le capteur réel. Le `.db3` ne contient pas les paquets Ethernet bruts : le SDK direct n'est donc pas la méthode naturelle pour relire un bag.

3 Deux méthodes d'accès

ROS 2

Bag ou LiDAR → `PointCloud2` → nœud Python/C++ → traitement.

ROS 2 est le meilleur choix pour le développement : il permet de rejouer les acquisitions, de tester exactement le même programme hors ligne et en temps réel, et de publier facilement les résultats. En Python, `sensor_msgs_py.point_cloud2.read_points` convertit le nuage en tableaux NumPy [**sensor_msgs_py**].

SDK C++ direct

LiDAR → UDP/série → SDK C++ → `PointCloudUnitree` et `LidarImuData`.

Le SDK fournit notamment `initializeUDP()`, `runParse()`, `getPointCloud()` et `getImuData()`. Cette solution réduit la chaîne logicielle, mais elle vise surtout le capteur réel et fait perdre la souplesse de relecture des bags. Le SDK officiel distribué est principalement prévu pour Linux ; aucune bibliothèque Windows native n'est actuellement fournie dans le dépôt.

4 Approche recommandée pour le temps réel

Le pilote ROS 2 officiel utilise déjà le SDK C++ puis publie le résultat en `PointCloud2` [**unitree_ros2**]. La chaîne conseillée est donc :

LiDAR → SDK Unitree → ROS 2 → projection/prétraitement → modèle de deep learning.

Le développement peut commencer en Python avec NumPy et PyTorch. Si les performances deviennent insuffisantes, la projection ou l'inférence peut être migrée vers un nœud ROS 2 C++. Le SDK direct ne devient nécessaire que si ROS 2 doit être supprimé du produit final ou si une mesure montre qu'il limite les performances.

5 Conclusion

Pour explorer la transformation des données en image de profondeur, il est préférable de conserver ROS 2 et de commencer en Python. Cette approche permet d'utiliser les enregistrements existants, puis de passer au temps réel sans changer l'algorithme. Le C++ ROS 2 constitue l'étape d'optimisation suivante ; le SDK C++ direct reste une option de déploiement plus spécialisée.